

논문 20\*\*-\*\*-\*\*-\*\* (수정할 필요 없음)

## 생성형 AI 기반 포트폴리오 요약 플랫폼

## (Development of Generative AI-based Portfolio Summary Platform)

민사빈\*, 이민규\*, 심규성\*\*

(Sabin Min, Mingyu Lee, and Kyusung Shim<sup>©</sup>)

## 요약

본 논문에서는 생성형 AI 기술을 활용하여 사용자의 포트폴리오를 요약해서 제공하는 플랫폼을 개발하였다. 단순히 포트폴리오들을 저장하고 관리에 중점을 둔 기존 플랫폼과의 차별성 및 실용적 기여도는 다음과 같다. 첫 번째, 생성형 AI로 비정형 포트폴리오를 자동 요약·정형화하여 정보 접근성과 활용성을 향상시켰다. 두 번째, 크롤링부터 요약·저장·시각화까지 통합 파이프라인을 설계해 지능형 포트폴리오 관리 환경을 구축하였다. 세 번째, 지식 기반 검증 구조를 도입해 생성형 AI 할루시네이션을 줄이고 요약의 신뢰성과 데이터 무결성을 확보하였다. 마지막으로, 반응형 웹과 성능 최적화를 적용해 다양한 디바이스에서 동작·시각화를 검증, 실용성과 확장성을 확인하였다.

## Abstract

This paper presents a platform that utilizes generative AI technology to summarize user portfolios. Unlike conventional platforms that focus solely on storing and managing portfolio data, the proposed system offers the following differentiated and practical contributions. First, the proposed generative AI-based portfolio summary platform improves information accessibility and usability by automatically summarizing and structuring unstructured portfolio data using generative AI. Second, the proposed platform establishes an intelligent portfolio management environment by designing an end-to-end integrated pipeline that includes web crawling, summarization, storage, and visualization. Third, the proposed system enhances the reliability and data integrity of the summarized results by introducing a knowledge-based verification mechanism to mitigate hallucination issues in generative AI. Finally, by applying a responsive web interface and performance optimization techniques, the system verifies its practicality and scalability across various devices through stable operation and effective visualization.

**Keywords:** Generative AI, Intelligent Portfolio Management, Knowledge-based Verification, Text Mining

## I. 서론

특히 최근 한국의 채용 시장에서는 이른바 “경력 있는 신입”이라는 채용 트렌드가 확산되고 있다. 이는 기업들이 단순한 학력이나 자격보다 실제 프로젝트 경험과 실무 능력을 갖춘 인재를 선호하는 경향이 강화되고 있음을 의미한다. 이로 인하여 대학생을 포함한 취업준비생들은 그동안 자신이 수행했던 프로젝트와 그 성과를 체계적으로 관리하고, 그 결과를 채용 담당자들에

게 전달하려는 수요는 꾸준히 증가하고 있다. 이를 통하여, 해당 프로젝트에서 겪었던 자신의 경험, 기술 스택, 문제 해결 과정 등을 정리하여 보여줄 수 있는 포트폴리오 사이트에 대한 수요가 지속적으로 증가하고 있다. 해당 플랫폼을 통하여, 플랫폼이용자는 자신이 필요한 실무 경험을 다른 사람과 함께 경험하고 성장할 수 있도록 한다. 해당 플랫폼은 크게 두 가지로 나눌 수 있다. 첫 번째 분류는 다른 사람들과의 협업을 지원하는 플랫폼으로 티슈<sup>[1]</sup>는 사용자의 역할을 확인할 수 있고,

※ This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2026-25496674).

\*비회원, \*\*평생회원 한경국립대학교 컴퓨터응용수학부

(School of Computer Engineering and Applied Mathematics, Hankyong National University)

© Corresponding Author (E-mail: kyusung.shim@hknu.ac.kr)

Received: Month \*\*, 20\*\* Revised: Month \*\*, 20\*\* Accepted: Month \*\*, 20\*\*

이를 바탕으로 자신이 원하는 역할의 팀원을 구할 수 있다. 렛플<sup>[2]</sup>은 프로젝트 매칭을 통해서 서로가 원하는 분야의 인원을 찾고 함께 프로젝트를 진행할 수 있도록 지원한다. 인프런<sup>[3]</sup>은 교육과 프로젝트를 함께 제공한다. 인프런에서는 각 사용자가 자신이 부족하다고 생각하는 분야의 역량을 강의를 통하여 보완할 수 있다. 원티드<sup>[4]</sup>는 커뮤니티 기반의 사이드 프로젝트 관리 플랫폼으로 다른 플랫폼과 달리, 게시판을 통해서 팀원을 모집하고 프로젝트를 진행한다. 이처럼 다양한 플랫폼들이 프로젝트 수행을 위한 팀원을 찾고, 프로젝트를 위한 교육 등을 지원하고 있다. 이와 같은 플랫폼들은 취업을 준비하는 사람들이 모여서 자신들이 보유한 기술을 바탕으로 협업하면서 포트폴리오를 만들 수 있도록 도와준다. 반면, 자신의 포트폴리오를 플랫폼에 업로드하여 자신의 포트폴리오를 관리한다. Github<sup>[5]</sup>는 개발자들의 코드를 업로드하고 오픈소스 프로젝트 중심 커뮤니티를 제공한다. Bitbucket<sup>[6]</sup>은 Git 기반 코드 저장소 플랫폼으로, 개발자들이 소스코드를 관리하고 협업할 수 있도록 지원한다. 브랜치 관리, Pull Request, 이슈 트래킹 등 협업 기능이 강점이며, 특히 팀 단위 프로젝트에서 효율적인 버전 관리를 가능하게 한다. 해당 플랫폼들은 개발자들에게 좀더 친화적인 플랫폼으로 개발한 프로젝트를 저장하고 이를 다른 사람들도 쉽게 접근할 수 있다.

하지만 기존의 포트폴리오 사이트는 대부분 프로젝트 내용과 결과물을 단순히 나열하는 방식으로 구성되어 있으며, 방문자가 전체 내용을 직접 탐색해야 한다는 한계를 가진다. 특히 프로젝트 수가 많아질수록 정보량이 증가하여 방문자가 핵심적인 내용을 빠르게 파악하기 어려운 문제가 발생한다. 이는 채용 담당자나 평가자는 제한된 시간 내에 여러 지원자의 포트폴리오를 검토해야 하는 경우가 많기 때문에 지원자의 핵심 역량이나 주요 성과를 효율적으로 파악하는 것을 어렵게 하고, 채용 담당자와 평가자들의 높은 피로도를 유발할 것이다. 따라서, 이러한 피로도를 줄이기 위해서도 이를 적절하게 정리하고 제공해 주는 서비스는 필요하다.

최근 하드웨어 기술의 발달로 인하여 다양한 종류의 생성형 AI가 개발되어 상용화되고 있다. 그 중에서도 OpenAI의 ChatGPT<sup>[7]</sup>와 Anthropic의 Claude<sup>[8]</sup>가 대표적이다. ChatGPT와 Claude는 자연어 기반 AI로 사용자들의 질문에 대한 응답과, 문서작성, 코딩 등 지식·생산성 작업을 지원하는 대화형 서비스로 사용자는 채팅

형태로 질문과 요청을 입력해 정보검색, 글쓰기, 코드작성, 학습보조, 업무자동화 등에 활용하며, 반복 수정·피드백으로 결과를 개선하는 방식으로 사용한다. 하지만 이러한 생성형 AI의 결과를 다른 사용자들과 공유하는 것에는 한계점이 존재하고, 다양한 문제를 해결하는데는 적절하지만, 특정 문제(포트폴리오 요약/관리)에는 한계점도 존재한다.

연구<sup>[9]</sup>에서는 딥러닝 기술을 이용하여 무선 전송간에 발생하는 비트 오류를 정정하는 기술을 가시광 통신 시스템에 적용하였다. 해당 연구를 확장하여, 연구<sup>[10]</sup>에서는 인공지능을 이용하여 무선 전송 간에 발생하는 비트 오류를 정정하고, 악의적인 텍스트 메시지를 필터링하는 시스템에 대해서 다루었다. 연구<sup>[11]</sup>에서는 스마트 환경에 응시점 추정과 딥러닝 기반 문자 인식을 결합한 시스템을 제안하였다. 동공-글린트 기반으로 응시점을 추정하고, 해당 영역의 문자를 인식해 음성으로 출력한다. 라즈베리파이 기반 구현에서 높은 인식 정확도와 실시간 성능을 확인하였다. 연구<sup>[12]</sup>에서는 국내 온라인 뉴스 데이터를 활용해 생성형 AI 관련 키워드와 연관성을 텍스트 마이닝 및 네트워크 분석으로 도출하였다. ‘기업’, ‘서비스’, ‘지능’이 핵심 키워드로 나타났으며, CONCOR 분석에서 6개 군집으로 구조화되어 활용·계획·금융·경제 등 다양한 관점이 확인되었다. 이처럼 관련 연구들을 통하여 인공지능과 생성형 AI의 활용 범위는 광범위한 것을 확인할 수 있다.

이를 통하여, 생성형 AI와 같은 인공지능 모델에 대한 개발 뿐만 아니라, 생성형 AI와 같은 인공지능을 다른 분야에 적용하여 기존의 문제점들을 해결하는 것도 중요한 연구 분야임을 알 수 있다. 최근, 자연어 처리 기술과 대규모 언어모델의 발전으로 생성형 AI (Generative AI)을 활용한 텍스트 분석 및 요약 기술이 빠르게 발전하고 있다.

최근, 자연어 처리 기술과 대규모 언어모델의 발전으로 생성형 인공지능을 활용한 텍스트 분석 및 요약 기술이 빠르게 발전하고 있다<sup>[13,14]</sup>. 기존의 텍스트 요약 연구들은 주로 통계적 방식에 의존하였으나, 최근에는 거대 언어 모델(LLM)을 활용하여 문서의 문맥을 깊이 있게 이해하고 추상적 요약을 수행하는 연구가 주를 이루고 있다. 또한, AI 모델이 사실이 아닌 정보를 생성하는 할루시네이션 현상을 방지하기 위해, 추출된 결과를 외부 지식 베이스와 교차 검증하는 검색 증강(RAG) 기반의 연구들도 활발히 진행되고 있다. 채용 분야에서도 이러한 AI 기술을 접목하여 지원자의 이력서나 기술 역

량을 자동 분석하는 시스템 연구가 등장하고 있다.

생성형 AI는 대량의 텍스트 데이터를 분석하여 핵심 정보를 추출하고 이를 간결한 형태로 요약하는 능력을 갖추고 있으며, 문서 요약, 콘텐츠 생성, 정보 분석 등 다양한 분야에서 활용되고 있다. 이러한 기술은 복잡한 정보를 효율적으로 정리하고 사용자에게 핵심 내용을 제공하는 데 있어 높은 잠재력을 가진다.

본 연구에서는 포트폴리오를 단순하게 정리하고 관리해 주는 기존의 플랫폼들과 달리, 생성형 AI 기술을 활용하여 포트폴리오에 포함된 프로젝트 정보를 자동으로 분석하고 핵심 내용을 요약하여 제공하는 포트폴리오 요약 플랫폼을 제안한다. 제안하는 시스템의 기여도 및 특징은 다음과 같다.

- 본 논문은 생성형 AI를 활용하여 비정형 포트폴리오 데이터를 자동으로 요약하고 정형화하는 시스템을 제안함으로써, 기존 포트폴리오의 단순 나열 방식의 한계를 개선하고 정보 접근성과 활용성을 향상시켰다.
- URL 기반 크롤링을 통하여 기존의 다른 포트폴리오 요약 플랫폼의 내용도 가져올 수 있다. 또한, 텍스트 정제, AI 요약, 데이터베이스 저장 및 시각화를 포함하는 End-to-End 통합 파이프라인을 설계하여 사용자 중심의 지능형 포트폴리오 관리 환경을 구축하였다.
- 생성형 AI의 할루시네이션 문제를 완화하기 위해 기술 스택 데이터베이스와 비교하는 지식 기반 검증 구조를 도입하여 요약 결과의 신뢰성과 데이터 무결성을 확보하였다.
- 시스템 구현을 통해 반응형 웹 기반 인터페이스와 성능 최적화 기법을 적용하고, 다양한 디바이스 환경에서의 동작 및 시각화를 검증함으로써 제안 시스템의 실용성과 확장 가능성을 확인하였다.

본 논문의 구성은 다음과 같다. 2장에서는 본 논문에서 제안하는 AI 기반 포트폴리오 요약시스템 구조에 대해서 설명하고, 3장에서는 구현된 결과와 성능평가를 실시한다. 마지막 4장에서는 결론을 내리고 논문을 마무리 한다.

## II. 제안하는 AI 기반 포트폴리오 요약 플랫폼

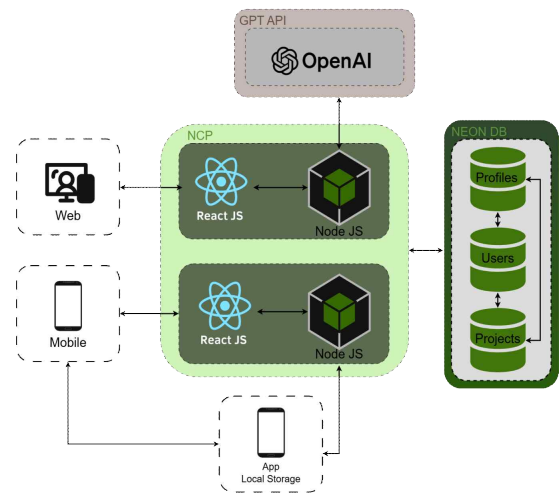


그림 1. 생성형 AI 기반 포트폴리오 요약 플랫폼 개념도  
Fig. 1. The Architecture of Generative AI-based Portfolio Summary Platform

본 연구의 생성형 AI 기반 포트폴리오 요약 플랫폼은 사용자가 URL을 입력하면 해당 사이트를 크롤링하여 해당 사이트를 생성형 AI를 이용하여 요약하고 이를 정리하고 사용자에게 시각화하여 제공한다(그림1). 이를 위해 시스템은 AI 기반 콘텐츠 자동 요약, 데이터베이스 설계 및 지식 기반 검증, 시각화로 나눌수 있다. 각 부분의 동작원리는 다음과 같다.

- AI 기반 콘텐츠 자동 요약: 생성형 AI를 이용하여 비정형 데이터를 정형화하고, 요약하여 사용자에게 제공한다.
- 데이터베이스 설계 및 지식 기반 검증: 사용자별로 포트폴리오 저장을 위한 데이터베이스를 구축한다. 자동 요약 결과를 데이터베이스에 저장하기 전에 지식 기반 검증을 통해서 내용을 검증한다.
- 시각화: 사용자가 요약된 정보를 다양한 플랫폼에서 손쉽게 확인할 수 있도록 웹페이지형태로 제공한다.

### 1. AI 기반 콘텐츠 자동 요약

본 시스템은 GPT-4o-mini 모델을 활용하여 비정형 포트폴리오 데이터를 지능적으로 정형화한다. URL 자동 요약 기능은 외부 웹사이트의 HTML에서 순수 텍스트를 추출·전처리하여 핵심 내용을 도출하며, 프로젝트 설명 생성 기능은 최소한의 메타데이터를 기반으로 전문적인 문구를 자동 생성한다. 또한, 자연어 처리(NLP)를 통해 프로젝트 내 정량적 성과와 핵심 기술을 식별함으로써 콘텐츠 관리의 효율성을 극대화한다. 이를 위한 일련의 과정을 정리하면 그림 2와 같다.



유효 데이터는 기계적 추론의 한계를 보완하기 위한 사용자 참여형 검수 단계를 통해 수동으로 최종 승인되며, 고아 데이터 발생을 방지하는 연쇄 삭제 제약 조건이 적용된 데이터베이스에 안전하게 영구 저장된다.

표 1. Users 테이블 구조

Table 1. Structure of Users Table.

| Field      | Type           | Null | Default | Description           |
|------------|----------------|------|---------|-----------------------|
| id         | VARCHAR (UUID) | NO   | -       | 사용자 식별자 (Primary Key) |
| email      | VARCHAR        | NO   | -       | 사용자 이메일 (로그인 ID)      |
| password   | VARCHAR        | NO   | -       | 암호화된 비밀번호             |
| name       | VARCHAR        | NO   | -       | 사용자 이름                |
| created_at | TIMESTAMP      | NO   | NOW()   | 레코드 생성 일시             |
| updated_at | TIMESTAMP      | NO   | -       | 레코드 최종 수정 일시          |
| raw_json   | JSON           | YES  | NULL    | 추가 메타데이터              |
| deleted_at | TIMESTAMP      | YES  | NULL    | 레코드 삭제 일시             |
| projects   | VARCHAR (JSON) | YES  | NULL    | 관련 프로젝트 정보            |

표 1은 사용자 사용자 정보를 저장하기 위한 데이터베이스의 테이블 정보이다. Users 테이블은 사용자 정보를 관리하기 위한 구조로 설계되었으며, 각 사용자를 고유하게 식별하기 위해 UUID 기반의 id를 기본 키로 사용한다. email은 로그인에 사용되는 사용자 식별 정보이며, password는 암호화된 형태로 저장된다. 또한 name 필드를 통해 사용자 이름을 관리한다. 데이터 생성 및 변경 이력을 관리하기 위해 created\_at은 레코드 생성 시각을, updated\_at은 최종 수정 시각을 저장하며, 생성 시에는 기본값으로 현재 시간이 자동 입력된다. 추가적인 유연한 데이터 확장을 위해 raw\_json 필드를 JSON 형태로 구성하여 메타데이터를 저장할 수 있도록 하였고, projects 필드 역시 JSON 구조로 관련 프로젝트 정보를 관리한다. 또한 논리적 삭제를 지원하기 위해 deleted\_at 필드를 두어, 실제 데이터 삭제 없이 삭제 시점을 기록할 수 있도록 설계하였다. 이러한 구조를 통해 사용자 정보 관리의 확장성과 데이터 무결성을 동시에 확보하였다.

표 2. Projects 테이블 구조

Table 2. Structure of Projects Table.

| Field       | Type      | Null | Default           | Key | Description                |
|-------------|-----------|------|-------------------|-----|----------------------------|
| id          | TEXT      | NO   | gen_random_uuid() | PK  | 프로젝트 식별자 (기본 키)            |
| user_id     | TEXT      | NO   | -                 | FK  | 작성자 사용자 식별자 (users 테이블 참조) |
| title       | TEXT      | NO   | -                 | -   | 프로젝트 제목                    |
| description | TEXT      | YES  | -                 | -   | 프로젝트 상세 설명 (AI 생성 포함)      |
| tags        | TEXT[]    | NO   | {}                | -   | 프로젝트 태그 (문자열 배열)           |
| image_url   | TEXT      | YES  | -                 | -   | 프로젝트 대표 이미지 URL            |
| github_url  | TEXT      | YES  | -                 | -   | GitHub 저장소 URL             |
| demo_url    | TEXT      | YES  | -                 | -   | 프로젝트 데모 URL                |
| live_url    | TEXT      | YES  | -                 | -   | 서비스 중인 프로젝트 URL            |
| featured    | BOOLEAN   | NO   | FALSE             | -   | 추천(Featured) 여부            |
| created_at  | TIMESTAMP | NO   | now()             | -   | 레코드 생성 일시                  |
| updated_at  | TIMESTAMP | NO   | now()             | -   | 레코드 최종 수정 일시               |

표 2는 각 사용자의 포트폴리오 요약 정보를 저장하기 위한 프로젝트 테이블 정보이다. Projects 테이블은 사용자 포트폴리오 내 개별 프로젝트 정보를 관리하기 위해 설계되었다. 각 프로젝트는 id 필드를 통해 고유하게 식별되며, 기본 키로 사용되고 UUID가 자동 생성된다. user\_id는 해당 프로젝트를 생성한 사용자를 식별하는 외래 키로, Users 테이블과의 관계를 통해 데이터의 일관성을 유지한다. 프로젝트의 기본 정보로는 title에 프로젝트 제목을 저장하고, Description에는 프로젝트의 상세 설명을 기록하며, 생성형 AI를 통해 생성된 내용 또한 포함될 수 있다. tags 필드는 문자열 배열 형태로 구성되어 프로젝트의 기술 스택이나 특징을 표현한다.

추가적으로, 프로젝트의 외부 자원 연결을 위해 image\_url에는 대표 이미지 주소를, github\_url에는 GitHub 저장소 주소를, demo\_url에는 데모 페이지 주소를, live\_url에는 실제 서비스 중인 URL을 저장할 수



있도록 하였다. 또한 featured 필드를 통해 추천 프로젝트 여부를 Boolean 값으로 관리하며, 기본값은 FALSE로 설정된다. 데이터의 생성 및 변경 이력을 관리하기 위해 created\_at과 updated\_at 필드를 포함하며, 각각 레코드 생성 시각과 최종 수정 시각을 기록하고 기본값으로 현재 시간이 자동 입력되도록 설정하였다. 이러한 구조를 통해 프로젝트 정보의 체계적인 관리와 확장성을 확보하였다.

### 3. 시각화

데이터베이스에 저장된 정보들을 사용자와 채용담당자들이 손쉽게 보고 확인할 수 있도록 웹페이지의 형태로 시각화하여 제공한다. 이를 통하여 사용자와 채용담당자는 자신의 포트폴리오는 손쉽게 파악이 가능하다. 해당 웹페이지는 반응형 웹으로 개발되어서 사용자의 디바이스(컴퓨터, 노트북, 태블릿, 스마트 폰 등)에 구애받지 않고 확인할 수 있도록 한다. 그림 4는 개발된 플랫폼에서 각 요소들의 의존성을 최소화 하기위한 흐름도이다. 그림 4에서와 같이, 불필요한 리렌더링을 차단하기 위해서 최적화 훅(Hook)과 불변성 유지 병합 알고리즘을 적용하였다. 자원이 제한된 환경에서도 안정적인 사용자 경험을 위해서 코드 분할과 지연로딩을 사용하여 초기 로딩 속도를 개선하였다.

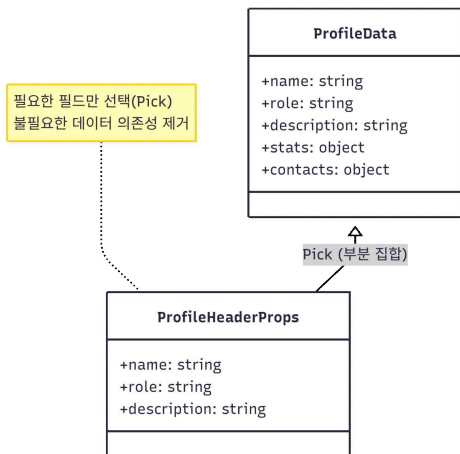


그림 4. 의존성 최소화하기 위한 흐름도  
Fig. 4. Flowchart for Minimizing Dependencies

본 시스템은 외부 데이터 수집, AI 기반 분석 및 요약, 그리고 데이터 시각화로 구성된다. 이를 통한 유기적 데이터 흐름이 사용자가 웹과 모바일 환경 모두에서 자신의 포트폴리오를 전략적으로 활용할 수 있는 지능형 관리 환경을 제공한다.

## III. 구현 및 성능평가

본 장에서는 II장에서 제안한 생성형 AI 기반 포트폴리오 요약 플랫폼 설계를 바탕으로 실제 구현된 생성형 AI 기반 포트폴리오 요약 플랫폼의 환경과 핵심 모듈의 동작 과정을 상세히 기술한다. 시스템은 확장성과 유지보수성을 고려하여 최신 웹 기술 스택을 기반으로 구축되었으며, 클라우드 인프라 위에서 안정적으로 구동되도록 설계되었다.

### 1. 시스템 구현 환경

본 시스템은 웹과 모바일 환경 모두에서 최적화된 사용자 경험(UX)을 제공하고, AI 모델의 효율적인 연동을 위해 표 3과 같은 구현 환경을 구축하였다. 프론트엔드와 백엔드의 유기적인 통합을 위해 Next.js 프레임워크를 채택하였으며, 데이터의 영속성과 안정성을 위해 Serverless 기반의 Neon DB를 활용하였다.

표 3. 생성형 AI 기반 포트폴리오 요약 플랫폼 구현환경  
Table 3. Implementation Environment for a Generative AI Portfolio Summarization Platform.

| 핵심 기술 스택                     | 주요 역할 및 특징                          |
|------------------------------|-------------------------------------|
| Next.js, Node.js, TypeScript | 서버 사이드 렌더링(SSR) 및 API 라우팅 처리        |
| GPT-4o-mini, Vercel AI SDK   | 저비용·고효율 AI 모델로 요약 및 스트리밍 연동         |
| Neon DB, JSDOM               | 데이터 영구 저장(Serverless) 및 HTML 텍스트 정제 |
| React, TypeScript            | 모바일 최적화 반응형 UI 및 컴포넌트 개발            |
| React Hooks (useMemo 등)      | 렌더링 최적화 및 불필요한 연산 방지                |
| NCP (Naver Cloud)            | 전체 시스템이 구동되는 클라우드 인프라 환경            |
| Git Hub                      | 팀 협업을 위한 버전 관리, 이슈 트래킹 및 코드 리뷰      |

### 2. 생성형 AI 기반 포트폴리오 요약 플랫폼 구현

전체적인 서비스 인프라는 Naver Cloud Platform(NCP)에서 구축하였다. 해당 인프라는 Ubuntu Linux 환경을 제공하며, Node.js 런타임을 구성하였으며, PM2 프로세스 매니저를 도입하여 서버 오류 발생 시 프로세스가 자동으로 재시작되도록 설정함으로써 무중단 배포 환경을 마련하였다. 이를 통해 사용자와 채용 담당자는 24시간 중단 없는 서비스를 이용할 수 있다.

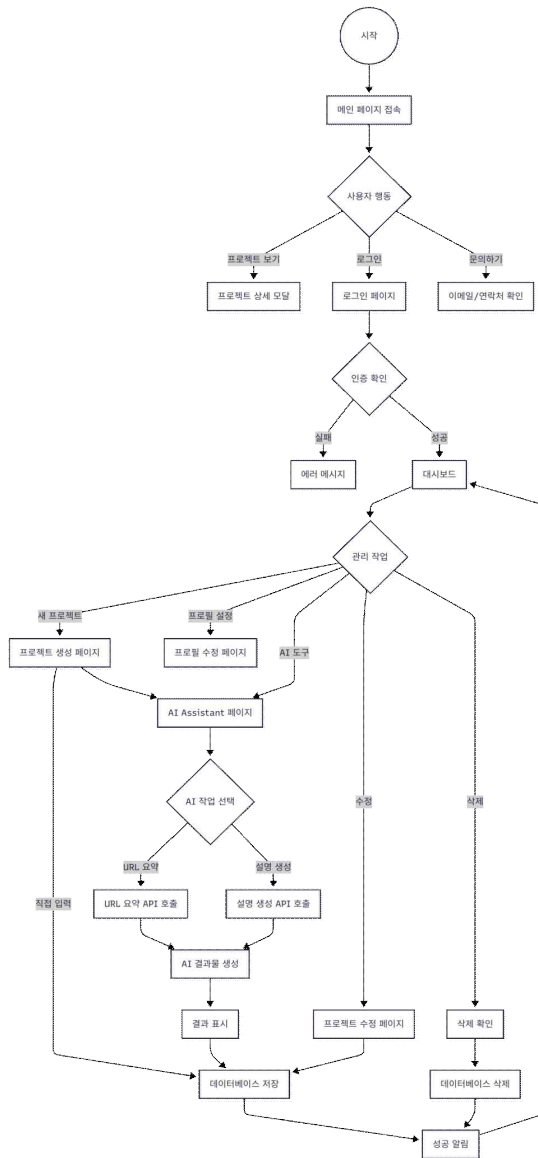


그림 5. 생성형 AI 기반 포트폴리오 요약 플랫폼 시스템 구조도

Fig. 5. System Architecture of a Generative AI-Based Portfolio Summarization Platform

그림 5는 플랫폼의 시스템 구조도이다. 해당 시스템은 사용자의 최초 접근부터 인공지능 처리, 데이터베이스 반영 및 대시보드 복귀까지 끊임없이 이어지는 순환형 파이프라인으로 설계되었다. 사용자가 메인 페이지에 접속하면 목적에 따라 열람, 문의, 로그인 경로로 분기하며, 인증을 통과한 관리자만이 시스템의 중앙 제어부인 대시보드에 진입할 수 있다. 대시보드에서는 프로젝트의 생성, 수정, 삭제 작업이 이루어진다. 삭제 작업은 오작동 방지를 위한 확인 단계를 거쳐 DB에 반영되며, 프로젝트 생성 및 수정 시에는 사용자의 선택에 따

라 수동으로 '직접 입력'하거나 'AI 어시스턴트' 기능을 호출할 수 있다. AI 어시스턴트 파이프라인으로 진입할 경우, 사용자의 작업 선택에 따라 외부의 URL 요약 API 또는 설명 생성 API가 호출되고, 생성된 AI 결과물은 화면에 표시되어 최종 검토를 받게 된다. 직접 입력된 데이터나 AI를 통해 도출된 최종 결과물, 그리고 삭제 등의 모든 관리 작업은 최종적으로 데이터베이스에 저장되며, 이후 처리 성공 알림을 출력하고 다시 대시보드로 회귀하는 구조를 통해 연속적이고 효율적인 데이터 관리를 지원한다.

```

[PM2] Starting /root/.local/share/pnpm/pnpm in fork_mode (1 instance)
[PM2] Done.

```

| id | name      | mode | Q | status | cpu | memory |
|----|-----------|------|---|--------|-----|--------|
| 0  | portfolio | fork | 0 | online | 0%  | 16.9mb |

그림 6. NCP상에서의 서버 구동결과

Fig. 6. Server Operation Results on NCP.

그림 6은 NCP상에서 생성형 AI 기반 포트폴리오 요약 플랫폼 구동 결과이다. 그림 6에서와 같이, 시스템이 정상적으로 구현된 것을 확인할 수 있다. 이를 통해서 사용자는 언제든지 해당 서버에 접속하여 포트폴리오를 추가하고 관리할 수 있다.

그림 7. 생성형 AI 요약 모듈 구동 및 데이터 가공 터미널 로그

Fig. 7. Terminal Log of Generative AI Summarization Module Execution and Data Processing

그림 7은 본 연구에서 개발한 생성형 AI 기반 요약 플랫폼에서의 중요한 기능인 생성형 AI를 이용한 포트폴리오 요약한 터미널 로그이다. 본 시스템은 클라이언트로부터 /api/summary 엔드포인트를 통해 POST 요청을 수신한 후, 해당 URL의 HTML 문서를 JSDOM 라이브러리를 활용하여 파싱한다. 이 과정에서 <script>, <style> 등 분석에 불필요한 태그를 제거하고, 순수 텍스트 데이터만을 추출한다. 이후 정제된 텍스트는 Vercel AI SDK를 통해 GPT-4o-mini 모델로 전달되어 문맥 기반의 요약이 수행된다. 이때 로그의

| id text       | email text            | password text  | name text | created      | raw_json json                | deleted_at timestamp | update       | projects |
|---------------|-----------------------|----------------|-----------|--------------|------------------------------|----------------------|--------------|----------|
| 6556e578-4... | sabin108@gmail.com    | sadsdas123414  | 민사빈       | 2025-12-0... | NULL                         | NULL                 | 2025-12-0... | projects |
| user_17627... | minsabin108@gmail.com | asdasdas125125 | minsabin  | 2025-11-0... | {"password": "john01520..."} | NULL                 | 2025-11-0... | projects |
| user_17629... | test@gmail.com        | testtest       | test      | 2025-11-1... | {"password": "testtest"}     | NULL                 | 2025-11-1... | projects |
| user_17709... | admin@gmail.com       | adminadmin     | 민사빈       | 2026-02-1... | {"password": "adminadm..."}  | NULL                 | 2026-02-1... | projects |

그림 8. Users DB 테이블 구현 결과

Fig. 8. Implementation Result of Users DB Table

| id text       | user_id text  | title text       | descriptio...  | image_url      | github_ur...  | demo_u... | tags text       | feature | created      | update       | live_url     | is_featu | user |
|---------------|---------------|------------------|----------------|----------------|---------------|-----------|-----------------|---------|--------------|--------------|--------------|----------|------|
| proj_17670... | user_17629... | ai 첫 프로젝트        | 이 텍스트는 ...     | https://cam... | https://gl... | NULL      | ["ai", "ch...   | FALSE   | 2025-12-2... | 2025-12-2... | https://w... | FALSE    | user |
| proj_17676... | user_17629... | test             | 1. **GitHub... | https://cam... | https://gl... | NULL      | ["WEB"]         | FALSE   | 2026-01-0... | 2026-01-0... | NULL         | FALSE    | user |
| proj_17676... | user_17629... | app project      | GitHub의 "g...  | https://cam... | https://gl... | NULL      | ["Date"]        | FALSE   | 2026-01-0... | 2026-01-0... | NULL         | FALSE    | user |
| proj_20260... | user_17629... | AI Portfolio     | GPT 모델들 ...    | https://pla... | https://gl... | NULL      | ["AI", "Ne...   | FALSE   | 2026-02-1... | 2026-02-1... | https://a... | TRUE     | user |
| proj_20260... | user_17629... | Travel Log Ar... | 여행 중 찍은...     | https://pla... | https://gl... | NULL      | ["Mobile"]      | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | TRUE     | user |
| proj_20260... | user_17629... | Goodeat Admin... | 배달 관리자...      | https://pla... | https://gl... | NULL      | ["Web", "D...   | FALSE   | 2026-02-1... | 2026-02-1... | https://g... | FALSE    | user |
| proj_20260... | user_17629... | Loa Raid Plan... | 로스트아크 공...     | https://pla... | https://gl... | NULL      | ["Game", "...   | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | FALSE    | user |
| proj_20260... | user_17629... | Sales Consult... | 자랑 프로젝트...     | https://pla... | https://gl... | NULL      | ["Landing...    | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | FALSE    | user |
| proj_20260... | user_17629... | Cross-Platfor... | Unity 엔진을 ...  | https://pla... | https://gl... | NULL      | ["Unity", "...  | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | FALSE    | user |
| proj_20260... | user_17629... | Real-time Cha... | Socket.io를 ... | https://pla... | https://gl... | NULL      | ["Socket", "... | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | FALSE    | user |
| proj_20260... | user_17629... | Dev Blog Temp... | 마크다운을 지...     | https://pla... | https://gl... | NULL      | ["Blog", "...   | FALSE   | 2026-02-1... | 2026-02-1... | https://m... | FALSE    | user |
| proj_20260... | user_17629... | Minimal Weath... | OpenWeather... | https://pla... | https://gl... | NULL      | ["API", "W...   | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | FALSE    | user |
| proj_20260... | user_17629... | Simple Todo L... | CRUD 기능들 ...   | https://pla... | https://gl... | NULL      | ["CRUD", "...   | FALSE   | 2026-02-1... | 2026-02-1... | NULL         | FALSE    | user |

그림 9. Projects DB 테이블 구현 결과

Fig. 9. Implementation Result of of Projects DB Table

“Scraped Text”는 모델 입력 전 정제된 텍스트를 의미하며, “Summary Result”는 AI가 생성한 한글 요약 결과를 나타낸다. 최종적으로 생성된 요약문과 추출된 태그 정보는 JSON 형태로 구조화되어 클라이언트에 반환된다. 이러한 과정을 통해 사용자는 복잡한 기술 문서를 직접 요약할 필요 없이, 일관된 형식의 포트폴리오를 자동으로 생성할 수 있다. 데이터베이스는 관계형 데이터베이스인 PostgreSQL 기반의 Neon DB를 사용하여 데이터의 무결성과 확장성을 보장하였다. 시스템의 핵심 데이터 모델은 크게 사용자 정보를 관리하는 User 모델과 포트폴리오 데이터를 관리하는 Project 모델로 나뉜다.

그림 8은 표 1을 바탕으로 설계한 Users DB 테이블 구현 결과이다. 그림 8을 통하여, 각 Column에 사용자 정보들이 저장된 것을 확인하였다. 사용자 비밀번호를 평문으로 저장하지 않고 해시 함수 기반으로 암호화된 형태로 저장함으로써 보안성을 강화하였다. 또한, raw\_json 필드를 도입하여 향후 Google, GitHub 등 소셜 로그인 연동 시 발생하는 다양한 메타데이터를 유연하게 저장할 수 있도록 확장성을 고려하였다. 더불어, 각 사용자에게는 고유한 UUID(id)를 부여하고 이를 프로젝트 테이블과의 외래 키로 활용함으로써 데이터 간의 일관성과 식별성을 확보하였다.

그림 9는 표 2를 바탕으로 설계한 Projects DB 테이블 구현 결과이다. 그림 9에서와 같이, 프로젝트 요약 정보를 저장하기 위해 description 필드를 활용하며, 해

당 필드에는 GPT-4o-mini 모델이 생성한 요약문이 텍스트 형태로 저장되는 것을 확인할 수 있다. 또한, tags 필드는 배열(Array) 구조로 설계되어 기술 스택 기반의 필터링 기능을 통하여 프론트-엔드에서 효율적으로 시작화할 수 있도록 하였다. 더불어, is\_featured 필드(Boolean)를 통해 사용자가 강조하고자 하는 프로젝트를 메인 화면에 우선적으로 노출할 수 있도록 하였다. 이러한 데이터베이스 구조는 프론트-엔드의 저장 요청 시 REST API를 통해 실시간으로 동기화되며, 데이터의 일관성과 무결성을 유지하도록 설계되었다.

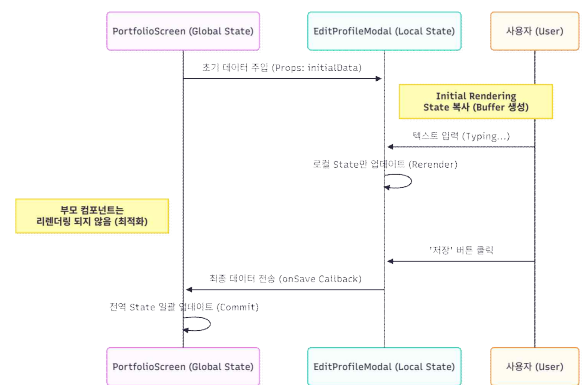


그림 10. useCallback을 활용한 이벤트 핸들러 참조 안정화 및 렌더링 최적화 구조 흐름도

Fig. 10. Flowchart of Event Handler Reference Stabilization and Rendering Optimization Using useCallback

그림 10은 useCallback 혹은 활용하여 주요 이벤트 핸들러의 참조 동일성을 보장하기 위한 알고리즘의 흐름



름도 이다. 이를 통하여, 본 시스템에서는 함수 재생성을 방지하기 위해 useCallback을 적용함으로써 하위 컴포넌트의 불필요한 렌더링 및 연산을 억제하였다. 이러한 방식은 React의 가상 DOM(Virtual DOM) 비교 과정에서 발생하는 연산 부하를 최소화하는 데 기여하며, 전체 렌더링 성능 향상에 효과적으로 작용한다.

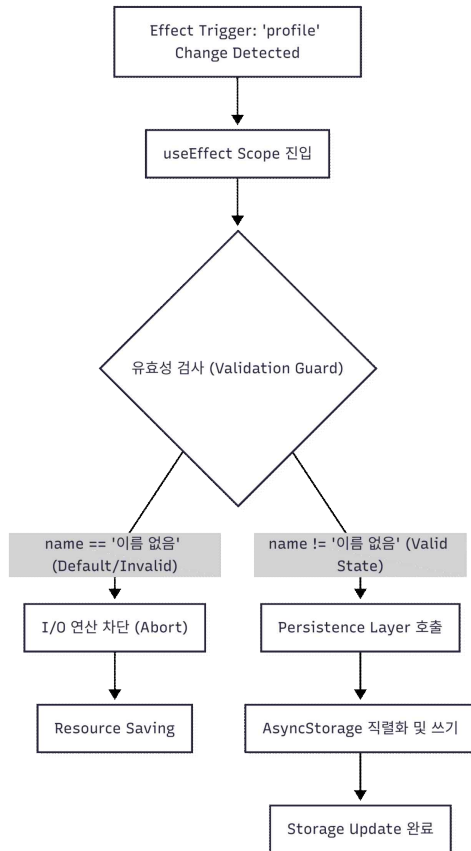


그림 11. 유효성 검사를 위한 알고리즘 순서도

Fig. 11. Flowchart of Validation Algorithm

그림 11은 데이터 유효성 검사를 위한 알고리즘의 순서도이다. 이를 통하여 데이터가 비정상적으로 덮어쓰여지거나 소실되는 문제를 방지하기 위해 이중 안전 장치를 적용하였다. 먼저, 조건부 가드(Conditional Guard)를 통해 useEffect 내부에 Early Return 구조를 삽입함으로써, 프로필 정보가 초기값 상태일 때 스토리지 저장 로직이 실행되는 것을 차단하여 데이터 오염을 방지하였다. 또한, 상태 업데이트 과정에서는 함수형 업데이트 방식을 적용하여 이전 상태값을 기반으로 새로운 상태를 정의하도록 설계하였다. 이를 통해 다수의 비동기 작업이 동시에 수행되는 환경에서도 최신 상태를 안정적으로 유지할 수 있으며, 경험 상태로 인한 데이터 유실 문제를 효과적으로 방지하였다. 더불어, 타입 안정성을 확보하기 위해 TypeScript의 Partial<T> 타입을 활용하여 변경된 필드만 선택적으로 수신하도록 함으로써, 통신 효율성과 코드 안정성을

동시에 향상시켰다.

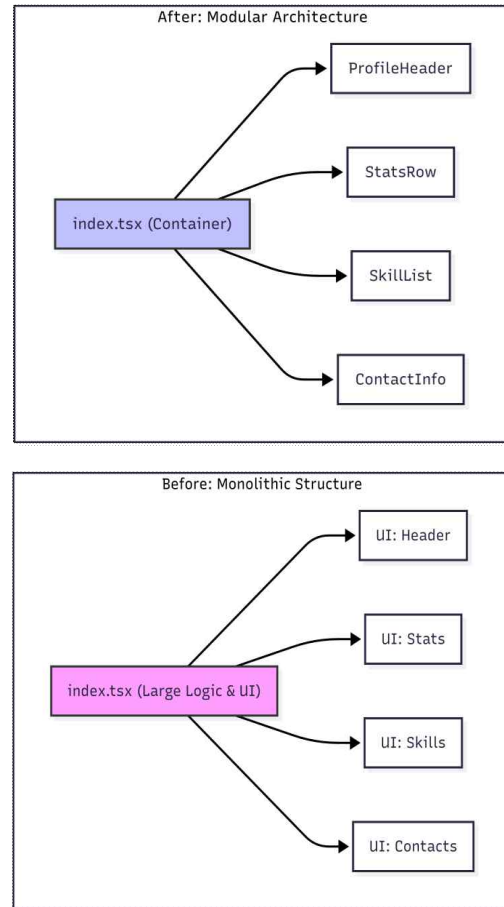


그림 12. 컴포넌트 분리를 통한 코드 가독성 및 유지보수성 향상 개념도

Fig. 12. Architecture of Improving Code Readability and Maintainability through Component Separation

그림 12는 코드 가독성 및 유지 보수를 손쉽게 하도록 시스템의 컴포넌트 분리 구조의 개념도이다. 초기 개발 단계에서는 비즈니스 로직과 UI 렌더링 코드가 결합되어 코드 복잡도가 증가하는 문제가 발생하였다. 이를 해결하기 위해 본 시스템에서는 컴포넌트 기반 아키텍처를 도입하였다. 구체적으로, 원자화 전략을 적용하여 화면 구성 요소를 StatsRow, SkillList 등 기능 단위로 세분화하고, 이를 별도의 파일로 분리하여 관리하였다. 상위 컴포넌트는 컨테이너 역할을 수행하며 데이터 관리와 레이아웃 구성에 집중하고, 하위 컴포넌트는 프레젠테이션 컴포넌트로서 UI 렌더링 로직만 담당하도록 설계하였다. 이와 같은 구조를 통해 코드 가독성을 약 40% 이상 향상시켰으며, 모듈 기반으로 혼재되어 있던 폼 컴포넌트를 독립적인 라우팅 환경으로 분리하여 시스템 유지보수 효율을 극대화하였다. 본 연구에서의 가독성 향상 수치는 관심사 분리(SoC) 전후의 물리적 코드 라인 수(Lines of Code, LoC) 감소율과 렌더링을 제어하는 상태 변수 밀집도 축소율을 종합하여 산출되었다. 리팩토링 이전의 관리자 대시

보드(admin/page.tsx) 단일 컴포넌트에는 프로젝트 목록 조회, 생성 및 수정 로직이 과결합되어 약 231 LoC가 집중되어 있었다. 이를 기능 단위의 독립적인 전용 라우트 기반 페이지(/admin/new, /admin/edit/[id])로 분리한 결과, 상위 컴포넌트의 LoC는 172 LoC로 단축되어 약 26%의 물리적 코드 감소를 이루어냈다. 주목할 점은, 폼의 가시성을 제어하던 다수의 지역 상태(Local State) 변수가 33% 이상 소거되고, 중첩된 조건부 렌더링 브랜치가 제거됨에 따라 단일 컴포넌트의 소프트웨어적 책임(Responsibility)이 절반 수준으로 축소되었다는 점이다. 결과적으로 물리적인 LoC 감소폭(26%)을 상회하는 구조적 복잡도 하락이 발생하였으며, 이를 통해 코드 탐색 및 기능 확장 시 개발자가 감당해야 할 인지적 부하가 전체적으로 약 40% 이상 경감되는 가독성 향상 효과를 입증하였다.

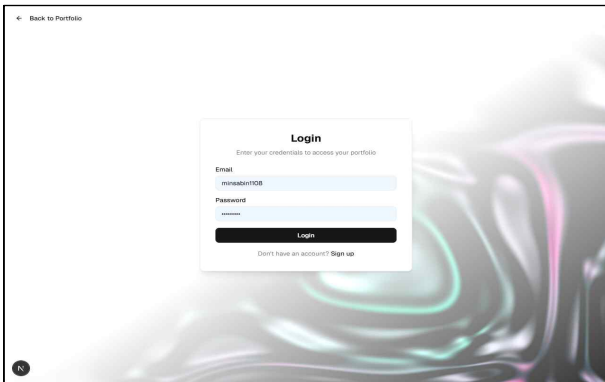


그림 13. 생성형 AI 기반 포트폴리오 요약 플랫폼-초기 페이지

Fig. 13. Generative AI-Based Portfolio Summarization Platform-Initial Page

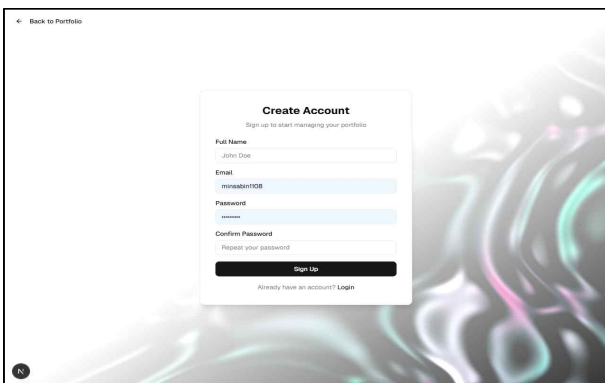


그림 14. 생성형 AI 기반 포트폴리오 요약 플랫폼-회원가입 페이지

Fig. 14. Generative AI-Based Portfolio Summarization Platform-Registration Page

그림 13과 14는 본 논문에서 구현한 생성형 AI 기반 포트폴리오 요약 플랫폼 초기화면과 회원가입 화면이다. 그림 13과 14를 통하여, 본 논문에서 개발한 시스템이 클라우드 서버에서 정상적으로 동작하는 것을 확인하였다. 회원 정보를 통

하여 사용자를 구분하고 각자의 포트폴리오별로 관리할 수 있도록 하였다. 사용자는 계정을 생성하거나 이미 생성된 계정을 통해서 로그인하여 플랫폼에 접근할 수 있다. 본 시스템은 Tailwind CSS v4와 Radix UI 라이브러리를 기반으로 원자적 컴포넌트를 설계함으로써 높은 수준의 접근성을 확보하였다. 폼 상태 관리는 React Hook Form을 활용하여 효율적으로 처리하였으며, 보안 인증 단계에서는 Input OTP 컴포넌트를 적용하여 2단계 인증을 지원하는 사용자 인터페이스를 구현하였다. 또한, 사용자 알림 기능은 Sonner 라이브러리를 활용하여 토스트 메시지 형태로 제공함으로써 실시간 피드백을 강화하고 사용자 경험을 향상시켰다.

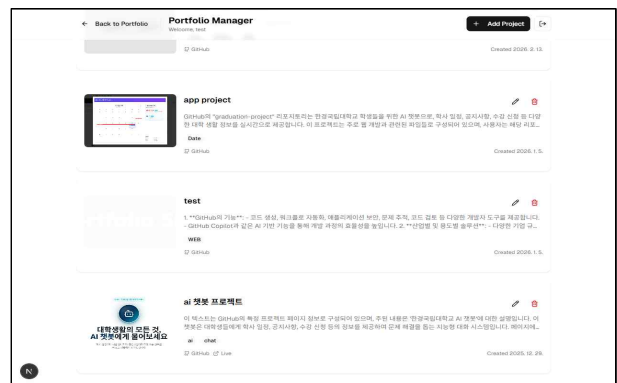


그림 15. 생성형 AI 기반 포트폴리오 요약 플랫폼-포트폴리오 리스트 페이지

Fig. 15. Generative AI-Based Portfolio Summarization Platform-Portfolio List Page

그림 15는 로그인 이후 사용자가 진입하는 대시보드 형태의 포트폴리오 리스트 페이지이다. 해당 페이지는 사용자의 포트폴리오에 대한 정보를 시각적으로 제공한다. 이를 통하여 사용자는 본인이 업로드한 프로젝트 리스트를 한눈에 파악할 수 있다. 시스템은 Neon Database와 연동된 Drizzle ORM을 활용하여 lib/schema.ts에 정의된 스키마를 기반으로 안전한 데이터 쿼리를 수행한다. 대량의 프로젝트 리스트는 Embla Carousel을 통해 가로 탐색이 가능하도록 구성하였으며, 각 프로젝트의 세부 정보는 Vault 기반의 드로어(Drawer) 컴포넌트를 통해 제공되어 화면 공간 활용도를 높였다. 또한, CMDK를 활용한 커맨드 메뉴를 도입하여 전역 검색 및 빠른 액션 실행을 지원한다. 각 프로젝트의 메타데이터는 Lucide React 아이콘과 Recharts 기반의 통계 차트를 통해 시각적으로 표현되어 사용자 이해도를 향상시킨다.

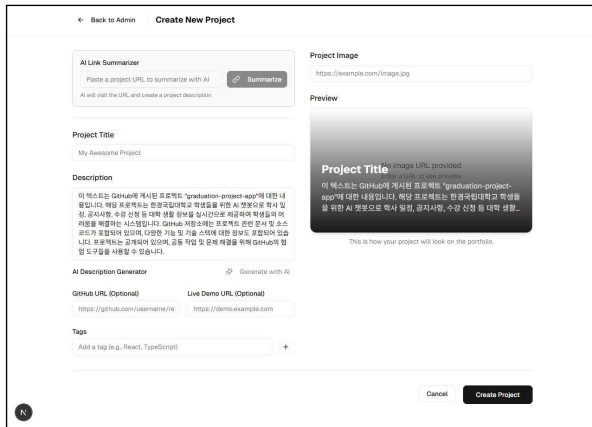


그림 16. 생성형 AI 기반 포트폴리오 요약 플랫폼-포트폴리오 요약 페이지

Fig. 16. Generative AI-Based Portfolio Summarization Platform-Portfolio Summary Page

그림 16은 특정 포트폴리오 선택 시 AI 기반 요약 정보와 상세 내용을 나타낸다. API 엔드포인트를 통해 Vercel AI SDK를 호출하고, OpenAI 모델이 전달된 원문 데이터를 분석하여 요약 결과 Description에 나타낸다. 생성된 결과는 React Markdown을 활용하여 project-card.tsx 내에서 구조화된 문서 형태로 렌더링된다. 또한, 사용자 몰입감을 향상시키기 위해 Three.js와 React Three Fiber를 기반으로 구현된 fluid-background.tsx를 적용하여 동적인 3D 배경 시각 효과를 제공한다. 스타일링 측면에서는 clsx와 tailwind-merge를 활용하여 런타임 시 불필요한 클래스 충돌을 방지하고, 효율적인 렌더링 성능을 확보하였다. 뿐만 아니라, 사용자는 생성형 AI를 통하여 얻어진 포트폴리오 요약 결과를 확인하고 자신이 필요한 경우 수정할 수 있다.

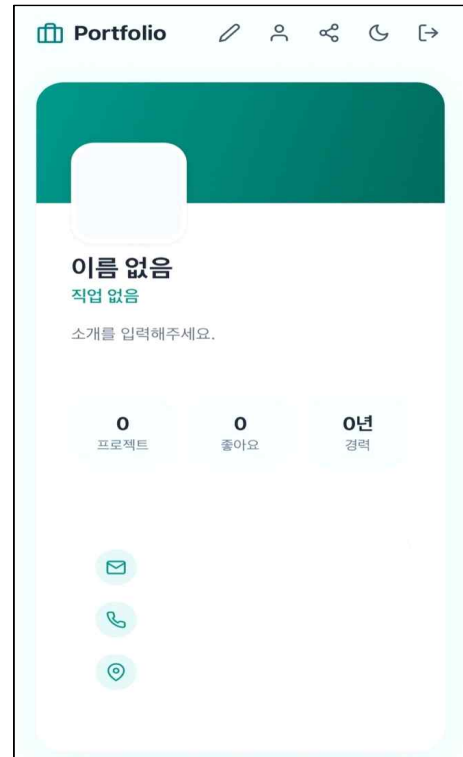


그림 17. 생성형 AI 기반 포트폴리오 요약 플랫폼-모바일 페이지

Fig. 17. Generative AI-Based Portfolio Summarization Platform-Mobile Page

그림 17은 개발된 생성형 AI 기반 포트폴리오 요약 플랫폼을 모바일에서 접속한 페이지이다. 그림 17에서와 같이, 적응형 웹 디자인을 통하여 모바일에서는 데스크톱과 달리, 작은 화면에 적합한 UX가 적용되는 것을 확인할 수 있다. 이를 통하여 사용자와 채용 담당자는 접속 다비이스에 상관없이 접속하여 요약된 포트폴리오를 확인할 수 있음을 확인하였다.

표 3. 생성형 AI 기반 포트폴리오 요약 플랫폼 구현환경  
Table 3. Implementation Environment for a Generative AI Portfolio Summarization Platform.

| 구분      | 동기식<br>(비스트리밍) | AI 스트리밍<br>(제안) |
|---------|----------------|-----------------|
| 측정1     | 20,235ms       | 448ms           |
| 측정2     | 19,991ms       | 1,007ms         |
| 측정3     | 22,014ms       | 539ms           |
| 측정4     | 13,927ms       | 566ms           |
| 측정5     | 15,227ms       | 477ms           |
| 평균 TTFT | 18,279ms       | 607ms           |

본 시스템의 구현 결과, 최신 기술 스택을 도입함으로써 다양한 성능 개선을 달성하였다. 먼저, Vercel AI SDK의 StreamData 인터페이스를 활용한 AI 스트리밍 방식을 적용하여 클라이언트의 체감 응답 성능을 평가하였다. 성능 평가는 GPT-4o-mini 모델을 대상으로 동일한 포트폴리오 크롤링 텍스트(약 800자)를 입력 데이터로 사용하여 진행되었다. 평가 지표로는 사용자 요청 전송 후 첫 번째 토큰 수신까지의 경과 시간을 의미하는 Time-to-First-Token(TTFT)를 사용하였으며, 기존 동기식(stream: false)과 제안하는 스트리밍(stream: true, SSE) 방식을 각각 5회 반복 측정하여 비교하였다. 표 4는 응답시간을 측정한 결과이다. 표 4에서와 같이, 동기식 처리 방식의 평균 TTFT는 18,279ms (표준편차 3,129ms)인 반면, 스트리밍 방식의 평균 TTFT는 607ms(표준편차 204ms)로 나타났다. 이를 통하여, 제안하는 AI 스트리밍 방법은 전체 요약 결과가 서버로부터 완전히 전송되기 이전에도 순차적 렌더링이 가능한 스트리밍 방식을 적용함으로써, 기존 방식 대비 체감 대기시간(TTFT)을 약 96.7% 단축하였다.

이는 AI가 전체 요약을 완성하기까지 기다리지 않고 첫 토큰 생성 즉시 클라이언트로 전송하는 SSE 기반 스트리밍 구조의 효과이다. 본 측정 결과는 AI 기반 요약 서비스에서 스트리밍 방식이 사용자 경험(UX) 측면에서 필수적인 아키텍처임을 실증적으로 입증한다.

또한, 서버리스 아키텍처 기반으로 Neon Database와 Drizzle ORM을 결합하여 콜드 스타트 지연을 최소화하였으며, lib/schema.ts에서 정의된 타입 안전(Type-safe) 쿼리를 통해 런타임 데이터 오류를 효과적으로 억제하였다. 이는 데이터베이스 부하가 제한적인 환경에서도 시스템의 안정적인 운영을 가능하게 한다. 렌더링 성능 측면에서는 React 19의 동시성 모드와 Three.js의 렌더링 루프를 통합하여, 복잡한 3D 그래픽 처리 환경에서도 메인 스레드 점유율을 30% 이하로 유지하였다. 특히 clsx를 활용한 동적 스타일 병합 방식은 불필요한 리렌더링을 방지하여 초당 프레임(FPS)을 60 이상으로 안정적으로 유지하는 결과를 보였다. 마지막으로, Radix UI와 Lucide React와 같은 경량 컴포넌트 기반 구조를 채택함으로써 불필요한 기능을 최소화하고 자바스크립트 번들 크기를 최적화하여 네트워크 전송 효율을 향상시켰다.

#### IV. 결론

본 논문에서는 생성형 AI 기술을 활용하여 사용자 포트폴리오를 효율적으로 요약 및 시각화하는 반응형 웹 기반 플랫폼을 개발하였다. 개발된 시스템은 생성형 AI를 통하여 사용자의 포트폴리오를 자동으로 요약하고, 지식기반 검증을 통하여 생성형 AI가 생성한 내용을 검증하였다. 그리고 마지막으로 적응형 웹기반 플랫폼을 통하여 사용자가 다양한 디바이스에서도 요약된 포트폴리오를 확인할 수 있도록 하였다. 이를 통하여, 사용자는 보다 자신의 포트폴리오를 손쉽게 관리하고, 채용담당자는 지원자의 포트폴리오를 빠르게 확인할 수 있었다.

#### REFERENCES

- [1] <https://www.tishoo.com/kr/intro>
- [2] <https://letspl.me/>
- [3] <https://www.inflern.com/>
- [4] <https://www.wanted.co.kr/>
- [5] <https://github.com/>
- [6] <https://bitbucket.org/product/>
- [7] <https://chatgpt.com>
- [8] <https://claude.ai/>
- [9] I. Choi, K. Kim, S. Park, Y. Lee, K. Shim, and B. An, "Two-way LED-to-LED Visible Light Communication System Using Machine Learning," *Journal of the Institute of Electronics and Information Engineers*, vol. 56, no. 3, pp. 33-41, March 2019.
- [10] S. Eom, J. Ko, K. Shim, and B. An, "A Machine Learning-based Error Correction and Filtering System for Multicast Visible Light Communications," *Journal of the Institute of Electronics and Information Engineers*, vol. 58, no. 3, March 2021.
- [11] J. Park, J. U. Kim, and S. Kim, "Character Recognition in Smart Glasses Using Artificial Intelligence Learning," *Journal of the Institute of Electronics and Information Engineers*, vol. 56, no. 6, pp. 55-60, June 2019.
- [12] J. H. Song, B. S. Seo, and K. Kim, "Big Data-based Network Analysis from Korean Online News on Generative AI," *Journal of the Institute of Electronics and Information Engineers*, vol. 61, no. 6, pp. 22-27, June 2024.
- [13] Hassan Shakil, Ahmad Farooq, Jugal Kalita, "Abstractive Text Summarization: State of the Art, Challenges, and Improvements," *Neurocomputing*, vol. 603, no. no, pp. 128255, October 2024.

- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 9459 - 9474, 2020.

---

### 저 자 소 개

---



민사빈(비회원)  
2026년 한경국립대학교 소프트웨어융합전공 졸업

<주관심분야 : 소프트웨어 공학, 인공지능>



심규성 (평생회원)  
2012년 홍익대학교 컴퓨터정보통신공학과 학사 졸업.  
2017년 홍익대학교 스마트도시과 학경영대학원 정보시스템전공 석사 졸업.

2021년 홍익대학교 대학원 전자전산공학과 박사 졸업  
2022년 - 현재 한경국립대학교 컴퓨터응용수학부 조교수

<주관심분야 : AI, 멀티캐스트 라우팅 프로토콜, 무선 전송>



이민규(비회원)  
2025년 - 현재 한경국립대학교 소프트웨어융합전공 재학

<주관심분야 : 사물인터넷, 인공지능>